



1 Introduction

A cipher transforms text one character at a time, which makes it a natural way to practise control flow. In this practical you will write five small programs that read text, transform it, and write the result, using loops, branches, and bit operations.

Every task is a complete program with its own `_start` entry point. You do not write or call functions, so you do not need the calling convention. You only need material from Lectures 1 to 4: registers and `mov`, the `read` and `write` system calls, ASCII conversion, branching, looping, and bit operations.

2 Building and running

Assemble with YASM and link with `ld`. For a file `task1.asm`:

```
yasm -f elf64 -g dwarf2 task1.asm    # produces task1.o
ld -o task1 task1.o                 # produces the executable task1
./task1                             # run it
```

A `makefile` is provided in the given files. Run `make` to build all five tasks and `make clean` to remove the generated files.

3 Submission and grading

Submit exactly these five files, with these names:

```
task1.asm task2.asm task3.asm task4.asm task5.asm
```

Each task is graded on its own, so you earn marks for every task you complete. Grading is automated. The marker runs your program, provides the test input on standard input, and compares your program's standard output to the expected output **byte for byte**. Output exactly what each task asks for and nothing else.

Plagiarism policy

It is in your own interest that you, at all times, act responsibly and ethically. As with any work done for the purpose of your university degree, remember that the University of Pretoria will not tolerate plagiarism. Do not copy another student's work or allow another student to copy yours. Doing so constitutes plagiarism, and apart from not gaining the intended experience, you may face disciplinary action as a result. See <http://www.library.up.ac.za/plagiarism/index.htm>.

4 Input and output rules

These rules apply to every task.

- All input arrives on standard input. All output goes to standard output.
- A “letter” means an ASCII letter, `a–z` or `A–Z`. Any other byte (space, digit, punctuation, newline) is not a letter.
- Unless a task says otherwise, every byte that is not a letter is copied to the output unchanged, including the trailing newline.
- The input is at most 4000 bytes.
- Every program must end by calling `exit` with status 0.

5 Tasks

Task 1: Swap case (3 marks)

Read the input and write it back with the case of every letter swapped: each lowercase letter becomes uppercase and each uppercase letter becomes lowercase. Non-letters are unchanged.

Input: `Hello, World!`
Output: `hELLO, wORLD!`

Task 2: Caesar cipher, shift 3 (5 marks)

Shift every letter forward by 3 places in the alphabet, wrapping around so that `x` becomes `a` and `Z` becomes `C`. Upper and lower case each wrap within themselves. Non-letters are unchanged.

Input: <code>abc XYZ!</code>	Output: <code>def ABC!</code>
Input: <code>Hello</code>	Output: <code>Khoor</code>

Task 3: Caesar cipher with a chosen shift (6 marks)

The first line of the input is the shift amount, a number from 0 to 25. The rest of the input is the text to encrypt. Apply a Caesar shift by that amount and write the encrypted text. Do not write the shift line itself.

Input: `5\nhello\n` Output: `mjqqt`

Here the shift is 5, so `hello` becomes `mjqqt`.

Task 4: XOR to hexadecimal (6 marks)

Combine every byte of the input with the key `0x2a` using `xor`. Print each resulting byte as two lowercase hexadecimal digits, with no spaces or separators, followed by a single newline at the end.

Input: `Hi` Output: `624320`

The input `Hi` followed by a newline is the three bytes `0x48 0x69 0x0a`. Each is combined with `0x2a` to give `0x62 0x43 0x20`, which print as `624320`. The newline byte is encrypted and printed like any other.

Task 5: Vigenère cipher (10 marks)

The first line of the input is a keyword made of lowercase letters. The rest is the text to encrypt. Shift each letter of the text forward by the amount given by the current keyword letter, where `a` shifts by 0, `b` by 1, and so on. Move to the next keyword letter for each letter you encrypt, wrapping back to the start of the keyword. Non-letters are copied unchanged and do not advance the keyword.

Keyword: `key` Text: `hello` Output: `rijvs`

The shifts from `key` are 10, 4, 24, then 10, 4 again, so `hello` becomes `rijvs`.

6 Mark distribution

Task	Mark
Task 1: Swap case	3
Task 2: Caesar cipher, shift 3	5
Task 3: Caesar cipher with a chosen shift	6
Task 4: XOR to hexadecimal	6
Task 5: Vigenère cipher	10
Total	30